



La bibbia delle Java socket

☰ Tags	OP
# Cap	-10
⚙ Status	In progress
🕒 Ultima modifica	@November 13, 2023 12:35 AM

[UDP Hello World](#)

[TCP Hello World](#)

▼ UDP Hello World

Server

```
import java.io.*;
import java.net.*;

public class UDPServer {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Usage: java UDPServer <port>");
            System.exit(1);
        }
    }
}
```

```

try {
    int port = Integer.parseInt(args[0]);

    if (port < 1024 || port > 65535) {
        System.out.println("Usage: java UDPServer <port>");
        System.exit(1);
    }
}
catch (NumberFormatException e) {
    System.out.println("Usage: java UDPServer <port>");
    System.exit(1);
}

DatagramSocket socket = null;

try {
    // Crea una socket UDP alla porta specificata
    socket = new DatagramSocket(Integer.parseInt(args[0]));
    System.out.println("UDPServer: in attesa di richieste");

    while (true) {
        // Buffer per ricevere i dati
        byte[] receiveData = new byte[1024];

        // Crea il pacchetto per ricevere i dati
        DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);

        // Attendi finché non ricevi un pacchetto
        socket.receive(receivePacket);

        // Ottieni i dati ricevuti e stampali
        String request = new String(receivePacket.getData());
        System.out.println("Messaggio ricevuto: " + request);

        // Esegui la logica del server e invia una risposta
        String response = "Hello from Server!";
        byte[] sendData = response.getBytes();
    }
}

```

```

        DatagramPacket sendPacket = new DatagramPacket(
            socket.send(sendPacket);
        System.out.println("Risposta inviata al cli
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    // Chiudi la socket se è stata aperta
    if (socket != null && !socket.isClosed()) {
        socket.close();
    }
}
}
}
}

```

Client

```

import java.io.*;
import java.net.*;

public class UDPClient {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Usage: java UDPClient <serv
            System.exit(1);
        }

        DatagramSocket socket = null;

        try {
            // Crea una socket UDP
            socket = new DatagramSocket();

            // Indirizzo IP del server
            InetAddress serverAddress = InetAddress.getByName
            int port = Integer.parseInt(args[1]);

```

```

        // Messaggio da inviare
        String message = "Hello from Client!";
        byte[] sendData = message.getBytes();

        // Crea un pacchetto da inviare al server
        DatagramPacket sendPacket = new DatagramPacket(

        // Invia il pacchetto
        socket.send(sendPacket);
        System.out.println("Messaggio inviato al server

        // Ricevi la risposta dal server
        byte[] receiveData = new byte[1024];
        DatagramPacket receivePacket = new DatagramPack
        socket.receive(receivePacket);

        // Ottieni i dati ricevuti e stampali
        String response = new String(receivePacket.get
        System.out.println("Risposta ricevuta dal serve

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        // Chiudi la socket se è stata aperta
        if (socket != null && !socket.isClosed()) {
            socket.close();
        }
    }
}
}

```

▼ TCP Hello World

Server

```

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.EOFException;

```

```

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class HelloWorldServer {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Usage: java HelloWorldServer <port>");
            return;
        }

        int portNumber;
        try {
            portNumber = Integer.parseInt(args[0]);

            if (portNumber < 1024 || portNumber > 65535) {
                System.out.println("La porta deve essere compresa tra 1024 e 65535");
                return;
            }
        } catch (NumberFormatException e) {
            System.out.println("Il parametro della porta deve essere un numero");
            return;
        }

        try (ServerSocket serverSocket = new ServerSocket(portNumber)) {
            System.out.println("Server in ascolto sulla porta " + portNumber);

            while (true) {
                try (Socket clientSocket = serverSocket.accept()) {
                    DataOutputStream out = new DataOutputStream(clientSocket.getOutputStream());
                    DataInputStream in = new DataInputStream(clientSocket.getInputStream());

                    String inputLine;
                    while ((inputLine = in.readUTF()) != null) {
                        System.out.println("Client dice: " + inputLine);
                        out.writeUTF("Hello, Client!");
                    }
                } catch (EOFException e) {
                    // Client disconnected
                }
            }
        }
    }
}

```

```

        System.out.println("Il client ha chiuso");
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

Client

```

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.Socket;

public class HelloWorldClient {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Usage: java HelloWorldClient <serverAddress> <port>");
            return;
        }

        String serverAddress = args[0];

        int portNumber;
        try {
            portNumber = Integer.parseInt(args[1]);
        } catch (NumberFormatException e) {
            System.out.println("Il parametro della porta deve essere un numero");
            return;
        }

        try (Socket socket = new Socket(serverAddress, portNumber)) {
            DataInputStream in = new DataInputStream(socket.getInputStream());
            DataOutputStream out = new DataOutputStream(socket.getOutputStream());

            out.writeUTF("HelloWorld");
            out.flush();

            String response = in.readUTF();
            System.out.println("Server response: " + response);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

        DataOutputStream out = new DataOutputStream(socket.getOutputStream());
        DataInputStream in = new DataInputStream(socket.getInputStream());

        String userInput;
        while (true) {
            // Legge l'input dall'utente
            userInput = System.console().readLine("Inserisci un comando: ");

            if (userInput.equals("exit")) {
                break;
            }

            // Invia l'input al server
            out.writeUTF(userInput);

            // Legge la risposta dal server
            String serverResponse = in.readUTF();
            System.out.println("Server dice: " + serverResponse);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```